

Deep Dive: Data Structures

Mutability, Trees, and Traversal

CMPT 211

Concordia University of Edmonton

Winter 2026

The Foundation: Mutable vs. Immutable

Before we choose a container, we must ask: "**Are we allowed to change the contents?**"

1. Mutable (The Whiteboard)

- You can write, erase, and rewrite.
- **Examples:** Lists [], Dictionaries {}, Sets {}.
- **Risk:** Accidental changes.

2. Immutable (The Stone Tablet)

- Once created, it is carved in stone.
- **Examples:** Tuples (), Strings "", Integers.
- **Benefit:** Safety & Speed.

1. Lists: The Workhorse

Structure: Ordered sequence of elements.

Memory: Python allocates extra space so you can add items later.

```
# A Mutable List
users = ["Alice", "Bob", "Charlie"]

# 1. Parsing (Accessing by Index)
first_user = users[0] # "Alice"

# 2. Modifying (Mutation)
users[1] = "Dave"      # Bob is gone!
users.append("Eve")    # List grows dynamically

print(users)
# Output: ['Alice', 'Dave', 'Charlie', 'Eve']
```

Traversing a List

How do we visit every item? Using a for loop.

```
grades = [85, 90, 78]

total = 0

# "g" becomes 85, then 90, then 78
for g in grades:
    print(f"Processing grade: {g}")
    total = total + g

print(f"Average: {total / len(grades)}")
```

2. Tuples: The Safety Vault

Structure: Ordered sequence, but **Locked**.

Why use them? To protect data integrity (e.g., GPS coordinates, Configs).

```
# An Immutable Tuple
gps_loc = (53.54, -113.49)

# 1. Parsing
lat = gps_loc[0]

# 2. Mutation Attempt
# gps_loc[0] = 54.00
# CRASH! TypeError: 'tuple' object does not support item assignment.
```

Think of Tuples as "Read-Only Lists".

3. Dictionaries: The Tree Structure

A Dictionary is not just a flat list. It is a **Tree**.

- **Root:** The main variable.
- **Branches:** The Keys.
- **Leaves:** The Values (which can be other dictionaries!).

```
student_system = {  
    "id": 101,                # Leaf  
    "profile": {              # Branch  
        "name": "Bernal",     # Leaf  
        "role": "Instructor"  # Leaf  
    },  
    "courses": ["CMPT211", "IT213"] # Branch -> List  
}
```

Parsing Nested Data (Climbing the Tree)

To get to the data, you must follow the path key by key.

```
data = {  
    "user": {  
        "details": {  
            "age": 45,  
            "city": "Edmonton"  
        }  
    }  
}  
  
# Path: user -> details -> city  
city = data["user"]["details"]["city"]  
print(city) # Output: Edmonton
```

The Rule: Reading left-to-right is like climbing down the tree.

Traversing a Dictionary

We can loop through Keys, Values, or Both.

```
scores = {"Math": 90, "Science": 80, "English": 85}

# The Best Way: .items() gives both Key and Value
for subject, score in scores.items():

    if score >= 90:
        print(f"{subject}: A Grade")
    else:
        print(f"{subject}: B Grade")

# Output:
# Math: A Grade
# Science: B Grade ...
```

Summary: Choosing the Right Tool

Structure	Mutability	Access Method	Best Analogy
List []	Mutable	Index [0]	Backpack
Tuple ()	Immutable	Index [0]	Sealed Box
Dict {}	Mutable	Key ["id"]	Tree / Filing Cabinet

Key Takeaway:

- Use **Lists** for collections of similar items (Grades, Files).
- Use **Dicts** for structured data (User Profiles, Settings).